VIETNAM NATIONAL UNIVERSITY - HCM
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT REPORT

DATA MINING – CO3029

Credit Card Fraud Detection: A Comparative Study of Machine Learning Models

---

Lecturer: Đỗ Thanh Thái

Group: 30 - L01

Student: Nguyễn Huy Hoàng - 2211093
Nguyễn Thanh Hoàng - 2211101
Nguyễn Đức Duy - 2210510

Source code: `https://github.com/huihoang/DataM-DataW`

Ho Chi Minh City - April 2026

Work Load

No.	Family Name	First Name	Student ID	Role	Percentage of work	Work Description
1	Nguyễn Huy	Hoàng	2211093	Leader	100%	Modeling & Evaluation & Conclusion & Visualization & Slide
2	Nguyễn Thanh	Hoàng	2211101		100%	Data Preprocessing & Pipeline & Visualization & Slide
3	Nguyễn Đức	Duy	2210510		100%	Data Understanding & Visualization & Introduction & Slide

Contents

List of Tables

List of Figures

1 Introduction

1.1 Problem

Background and Technological Trends

In recent years, the rapid growth of digital payment systems and online transactions has significantly increased the risk of fraudulent activities. With the widespread use of credit cards and e-commerce platforms, financial fraud has become a critical issue affecting both individuals and financial institutions.

Traditional rule-based fraud detection systems are no longer sufficient due to the increasing complexity and volume of transactions. Therefore, data mining and machine learning techniques have become essential tools for detecting fraudulent behavior efficiently and accurately.

Why Choose This Topic?

The idea behind this project is to apply data mining techniques to analyze transaction data and identify patterns that distinguish fraudulent transactions from legitimate ones

This topic is chosen because:

- Fraud detection is a real-world, high-impact problem in financial systems, where undetected fraud can lead to significant economic losses.
- The dataset is large-scale and reflects realistic transaction behavior, making it suitable for practical analysis.
- It provides an opportunity to apply a complete data mining pipeline, including exploratory data analysis (EDA), preprocessing, and machine learning modeling.
- The problem involves severe class imbalance, which is a common and challenging issue in real-world applications.

1.2 Goal

The main objectives of this project are:

- To explore and understand the structure and characteristics of the transaction dataset.
- To identify patterns and key factors that distinguish fraudulent transactions from normal ones.
- To address the class imbalance problem using appropriate techniques (e.g., resampling methods).
- To build and compare multiple machine learning models for fraud detection.
- To evaluate model performance using suitable metrics, with a focus on Recall and PR-AUC.
- To analyze the impact of different decision thresholds on model performance.

1.3 Scope

This project focuses on:

- **Dataset:** `fraudTrain.csv`, containing transaction records for fraud detection.
- **Tasks:**
  - **Data Understanding:** Analyze data distribution, detect imbalance, and extract initial insights.
  - **Data Preprocessing:** Perform data cleaning, scaling, dimensionality reduction, and imbalance handling (e.g., SMOTE).
  - **Modeling & Evaluation:** Train and compare four models (Logistic Regression, Decision Tree, Gaussian Naive Bayes, and ANN/MLP) on both imbalanced and SMOTE-balanced datasets, evaluate under multiple thresholds (0.7, 0.5, 0.2), and assess performance using metrics such as Accuracy, Precision, Recall, F1-score, ROC-AUC, and PR-AUC.

2 Data Understanding

2.1 Data Overview

The dataset used in this project contains financial transaction records, including both legitimate and fraudulent activities.

General Information

- Number of records: 1,296,675
- Number of features: 23
- Target variable: is_fraud

Fraud Analysis

- Fraud count: 7,506
- Fraud ratio: 0.005789 (0.58%)

This indicates a severe class imbalance problem, where fraudulent transactions are extremely rare compared to normal transactions.

2.2 Feature Description

The dataset consists of multiple types of features, which can be grouped as follows:

2.2.1 Identification Features

- stt: Transaction index
- cc_num: Customer credit card number
- trans_num: Unique transaction identifier

Observation: These features serve as identifiers and do not provide meaningful predictive information for fraud detection.

2.2.2 Time Features

- trans_date_trans_time: Timestamp of the transaction
- unix_time: Transaction time in Unix timestamp format

Applications:

- Extract temporal features such as:
  - hour of the transaction
  - day of the week
- Detect abnormal transaction timing patterns

2.2.3 Transaction Features

- amt: Transaction amount
- category: Type of transaction (e.g., shopping, dining, entertainment)
- merchant: Merchant name

Observation:

- The transaction amount (amt) is a critical feature
- The category helps identify spending behavior patterns

2.2.4 Customer Information

- first, last: Customer name
- gender: Gender of the customer
- dob: Date of birth
- job: Occupation

Applications:

- Derive age from date of birth
- Analyze demographic-based transaction behavior

2.2.5 Customer Location

- street, city, state, zip: Customer address
- lat, long: Geographic coordinates of the customer
- city_pop: Population of the customer's city

Applications:

- Detect unusual transaction locations
- Combine with merchant location to compute distance

2.2.6 Merchant Location

- merch_lat, merch_long: Geographic coordinates of the merchant

Applications:

- Calculate transaction distance:

$$\text{distance} = \text{distance}(\text{customer location}, \text{merchant location})$$

- Identify suspicious transactions occurring far from the customer's usual location

2.2.7 Target Variable

- is_fraud: Fraud label
  - 0 = Normal transaction
  - 1 = Fraudulent transaction

This is the target variable (label) used for classification.

2.3 Data Quality

- No missing values detected
- Data types include:
  - Numerical: float64, int64
  - Categorical: string
- Memory usage: approximately 227 MB

2.4 Key Observations

- The dataset is large-scale, making it suitable for machine learning applications
- The data is clean, with no missing values
- There is a strong class imbalance problem
- The dataset contains many categorical and temporal features, which are valuable for feature engineering

2.5 Visualization and Interpretation

This section presents key visualizations of the dataset along with their interpretations.

2.5.1 Class Distribution

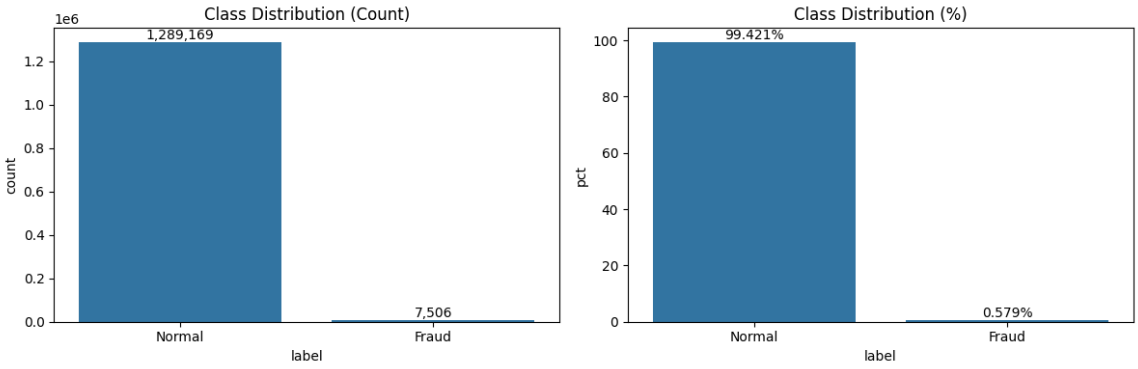

Figure 1: *Class Distribution of Transactions (Count and Percentage)*

Analysis:

- The dataset is highly imbalanced, with normal transactions accounting for approximately 99.42% of the data.
- Fraudulent transactions represent only about 0.58%.
- This imbalance indicates that classification models may be biased toward the majority class.
- Special techniques such as resampling or class weighting are required.

2.5.2 Transaction Amount Distribution

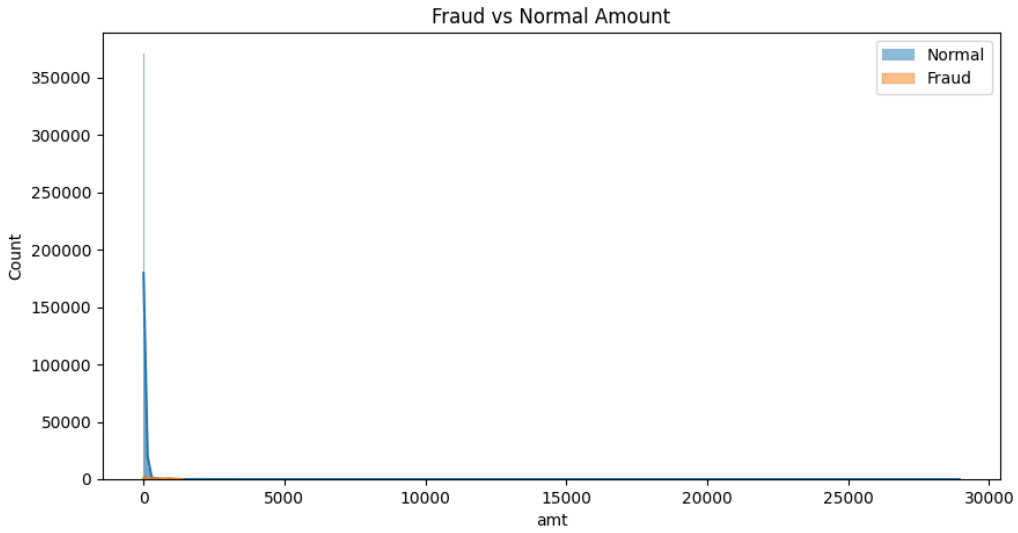

Figure 2: *Distribution of Transaction Amounts for Normal vs Fraudulent Transactions*

Analysis:

- The distribution of transaction amounts is highly skewed toward smaller values.
- Fraudulent transactions tend to have higher amounts compared to normal transactions.
- There is overlap between the two classes, indicating that transaction amount alone is insufficient for classification.
- The presence of extreme values suggests the existence of outliers.

2.5.3 Time-Based Fraud Distribution

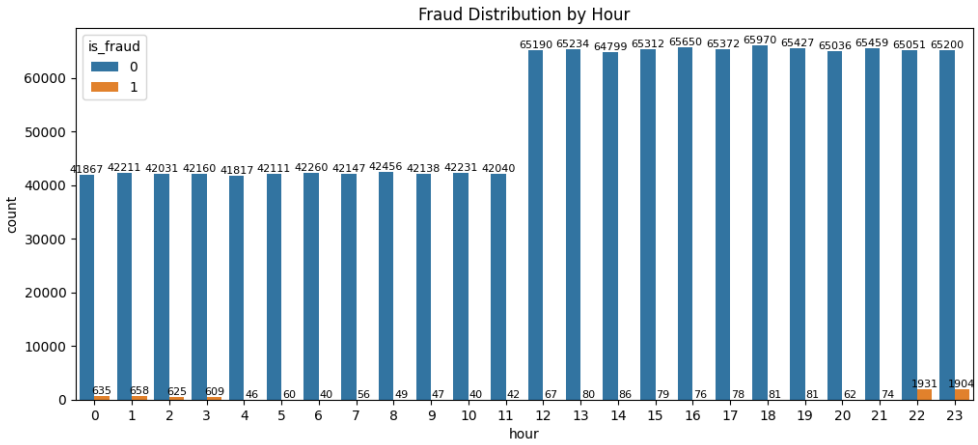

Figure 3: *Fraud Distribution by Hour of the Day*

Analysis:

- Transactions occur throughout all hours of the day.
- Fraudulent activities are present at every hour but remain relatively sparse.
- A slight increase in fraud can be observed during late hours.
- This suggests that temporal features may help improve fraud detection performance.

2.5.4 Category-Based Fraud Rate

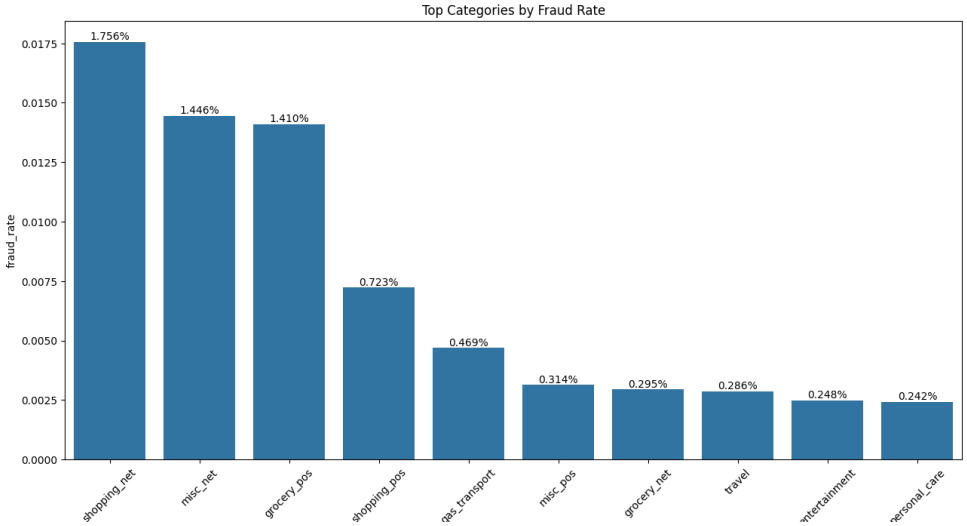

Figure 4: *Top Transaction Categories by Fraud Rate*

Analysis:

- Certain categories, particularly online-related categories (e.g., shopping_net), exhibit higher fraud rates.

- Fraudulent activities are more prevalent in e-commerce transactions.
- This highlights the importance of the category feature in fraud detection.

2.5.5 Correlation Matrix

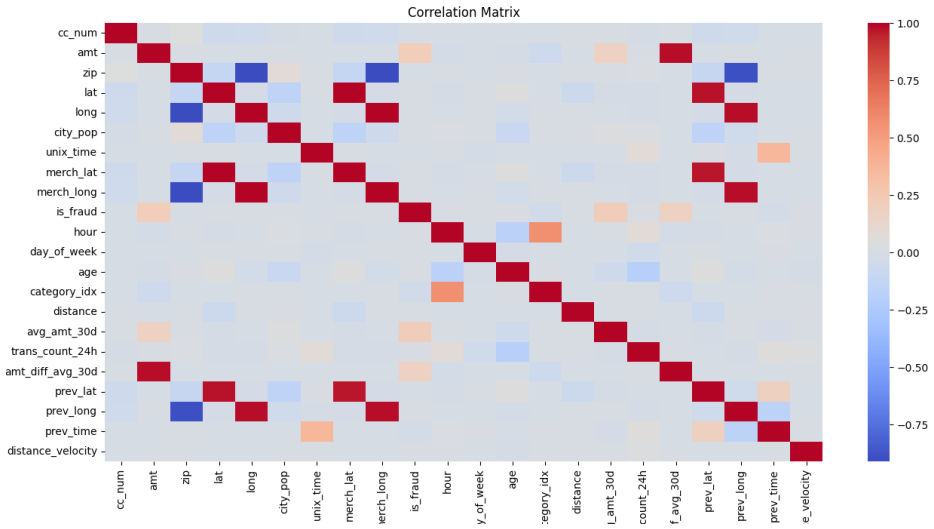

Figure 5: *Correlation Matrix of Numerical Features*

Analysis:

- Strong correlations are observed between geographic features such as latitude and merchant latitude, as well as longitude and merchant longitude.
- Most features show weak correlation with the target variable.
- This indicates that fraud detection is a complex and non-linear problem.
- Advanced machine learning models are required to capture these relationships.

2.5.6 Fraud Analysis by State

To better understand geographical patterns in fraudulent transactions, the fraud rate was analyzed across all states (with at least 100 transactions to ensure statistical reliability).

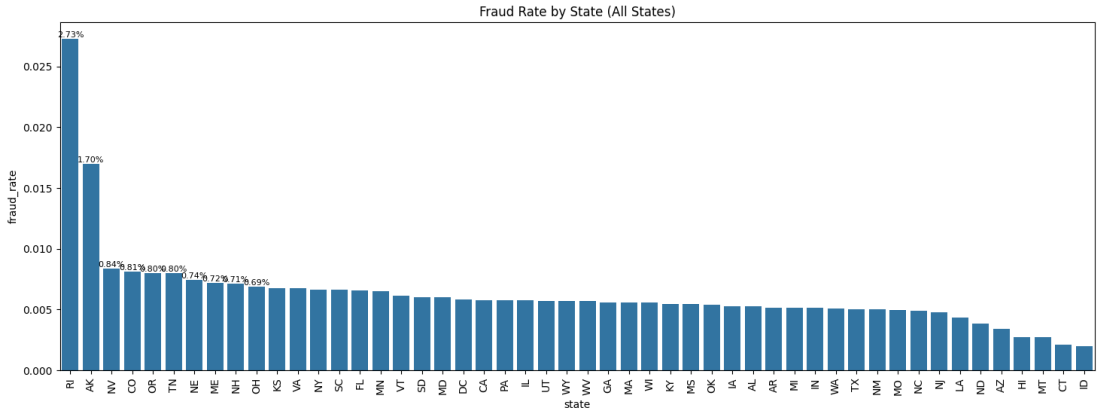

Figure 6: *Fraud rate by state*

Key Observations:

- Fraud rates vary significantly across states, ranging from approximately **0.20% to 2.73%**.
- The state with the highest fraud rate is:
  - **RI (Rhode Island)**: 2.73% (15 fraud cases out of 550 transactions)

  However, this state has a relatively small number of transactions, which may lead to higher variance and less reliable estimates.
- Other states with relatively high fraud rates include:
  - AK (1.70%)
  - NV (0.84%)
  - CO (0.81%)
  - OR (0.80%)
- Most states have fraud rates clustered around **0.5% to 0.7%**, which is close to the overall dataset average (0.58%).
- States with the lowest fraud rates include:
  - ID (0.20%)
  - CT (0.21%)
  - MT (0.27%)
  - HI (0.27%)
- Large states with high transaction volumes such as:
  - CA (0.58%)

- TX (0.50%)
- NY (0.66%)

tend to have fraud rates close to the global average, suggesting more stable and representative behavior.

Conclusion:

- Fraud behavior shows geographical variation, but extreme values often occur in states with low transaction counts.
- Location-based features (e.g., state) can contribute to fraud detection but should be combined with other features to avoid overfitting to small regions.

2.5.7 Fraud Analysis by Gender

Fraud distribution was also analyzed based on customer gender.

Figure 7: *Fraud rate by gender*

Statistics:

- Female:
  - Fraud cases: 3,735
  - Total transactions: 709,863
  - Fraud rate: 0.526%

- Male:
  - Fraud cases: 3,771
  - Total transactions: 586,812
  - Fraud rate: 0.643%

Key Observations:

- Male customers have a slightly higher fraud rate than female customers:
  - Difference: approximately **0.12%**
- Although the difference exists, it is relatively small, indicating that gender alone is not a strong predictor of fraud.
- Female customers have more total transactions, but a lower fraud rate, suggesting more stable transaction behavior overall.

Conclusion:

- Gender has a minor impact on fraud likelihood.
- It can be used as a supporting feature in modeling but should not be relied upon as a primary indicator.

3 Data Preprocessing

3.1 Overview

Our preprocessing workflow follows these key steps:

1. **Load data and split**: Load from separate train/test CSV files and split training set chronologically to simulate real-world conditions
2. **Data cleaning**: Validate data quality, confirm no missing values, duplicates, or corruption issues
3. **Feature engineering**: Create features from temporal (hour, day, month), geographic (distance), and demographic dimensions (age), and compute behavioral statistics from merchant/cardholder history
4. **Feature selection**: Use multiple ranking methods to identify top features with highest discriminative power
5. **Data finalization**: Balance training classes with SMOTE and prepare datasets for modeling

3.2 Data Loading and Splitting

The first step in preprocessing is loading data from two separate CSV files: fraudTrain.csv contains historical transactions with fraud/non-fraud labels, and fraudTest.csv is the holdout set for final evaluation. Data splitting strategy is critical in fraud detection because fraud patterns evolve over time—a model trained on future data will not generalize well to past data.

For this reason, we use **chronological split** instead of traditional random split. This approach preserves temporal order and ensures the training set always precedes validation/test sets, accurately simulating production conditions where we train on historical data and deploy to predict future transactions.

The raw dataset is split using **chronological split** to minimize information leakage:

- **fraudTrain.csv**: Contains 1,296,675 transaction rows (75% of total) with fraud/non-fraud labels, spanning a time range to provide diverse samples
- **fraudTest.csv**: Contains 555,719 transaction rows (25% of total) with timestamps after fraudTrain, used for final evaluation on unseen data
- **Training split**: From fraudTrain, we allocate 80% (1,037,340 rows) for model training and 20% (259,335 rows) for validation to tune hyperparameters, all split chronologically by timestamp

Reasons for using chronological split:

- **Fraud patterns evolve**: Fraud is not a static phenomenon—fraudsters continuously change tactics and methods. Training on current data but testing on future data (or vice versa) produces unreliable results
- **Realistic production simulation**: In practice, we train models on transactions from the past (2-3 months prior) and deploy to predict today's transactions. Chronological split accurately reproduces this scenario

- **Avoid temporal leakage**: Random split might accidentally let validation sets see merchant patterns from future dates in the training set, making models perform better than they actually should

Table 1: *Summary of original data after chronological split*

Dataset	Rows	Columns	Fraud Rate	Notes
train_split	1,037,340	30	0.166%	80% of fraudTrain
validation_split	259,335	30	0.155%	20% of fraudTrain
test_holdout	555,719	30	0.080%	fraudTest

3.3 Data Cleaning and Quality Checks

Before creating any features, we must ensure the raw data is clean and reliable. This step includes checking the target variable, duplicate rows, missing values, and analyzing column cardinality. These basic checks are the foundation for all subsequent steps.

3.3.1 Target Variable Check

The target column *is_fraud* is a binary variable (0 = legitimate transaction, 1 = fraud). This target has clear meaning: fraud=1 when merchants or banks confirm the transaction as fraudulent after being reported. It is crucial to recognize that this dataset is highly imbalanced with fraud comprising only 0.13% overall—a major challenge for learning models since they will tend to be biased toward predicting the majority class (legitimate). We will address this issue later using SMOTE resampling.

Table 2: *Class distribution details (Fraud vs Non-fraud)*

Dataset	Label	Rows	Rate %	Notes
Train	Legitimate (0)	1,035,623	99.834%	Majority class
	Fraud (1)	1,717	0.166%	Minority class
Validation	Legitimate (0)	259,103	99.911%	Majority class
	Fraud (1)	232	0.089%	Minority class
Test	Legitimate (0)	555,274	99.920%	Majority class
	Fraud (1)	445	0.080%	Minority class

3.3.2 Duplicate Rows

Duplicate rows (transactions identical across all columns) can occur from data entry errors, system failures, or fraud attempts (e.g., retrying the same transaction multiple times). This check confirms there are no completely duplicate rows in the dataset—this is good because it reduces noise and ensures each observation is unique.

Table 3: *Duplicate Rows Check*

Dataset	Duplicate Count	Rate %
Train	0	0.00%
Validation	0	0.00%
Test	0	0.00%

3.3.3 Missing Values

Missing values (NaN, NULL) are common issues in real-world datasets. They can arise from:

- Data entry errors or system failures
- Values not applicable (e.g., zip code for international merchants)

- Information not yet captured or deleted

This check confirms the dataset has no missing values, which is excellent because it reduces the need for complex imputation (mean imputation, forward fill, etc.) and ensures no information loss.

Table 4: *Missing Values Check*

Dataset	Total Missing Values	Rate %
Train	0	0%
Validation	0	0%
Test	0	0%

3.3.4 Cardinality Analysis

Cardinality Analysis determines the number of unique values in each column—this metric helps us decide which columns to keep, remove, or transform.

- **High cardinality ($> 10,000$ or 100%)**: Columns like cc_num (credit card number), trans_num (transaction ID), first/last name have very high cardinality. These features are typically identifiers rather than predictive features—keeping them will cause model overfitting (memorizing training data rather than learning patterns)
- **Medium cardinality (10-1000)**: Columns like merchant, state, category have medium cardinality. Some can be kept or transformed: merchant can be replaced with statistics like fraud rate, state can be kept for categorical features
- **Low cardinality (< 10)**: gender, is_weekend are perfect candidates to keep—they are already useful features

Table 5: *Cardinality Analysis (Number of unique values)*

Column	Unique Values	Unique Rate	Treatment
cc_num	4,112	0.40%	Remove (identifier)
merchant	2,906	0.28%	Replace with merchant_fraud_rate
trans_num	1,555,456	100%	Remove (unique identifier)
first	>1,000,000	Very high	Remove (proper name)
last	>1,000,000	Very high	Remove (proper name)
state	51	Low	Keep for categorical
category	14	Low	Keep for categorical
gender	2	Very low	Keep

3.4 Exploratory Data Analysis (EDA)

After initial data cleaning, we perform comprehensive exploratory data analysis to understand patterns, distributions, and key insights that will guide our feature engineering process.

3.4.1 Class Imbalance Analysis

The primary challenge in this dataset is extreme class imbalance:

Key observations:

- **Fraud represents <0.2% of transactions**—this is realistic but creates severe imbalance

Table 6: *Class Distribution in Processed Datasets*

Partition	Legitimate (0)	Fraud (1)	Fraud Rate
Train Split	1,035,623	1,717	0.166%
Validation Split	259,103	232	0.089%
Test Holdout	555,274	445	0.080%

- **Standard ML models struggle**: Algorithms tend to overfit the majority class
- **Accuracy is misleading**: A model predicting "never fraud" achieves >99.8% accuracy
- **Solution**: Use metrics like Recall, Precision, F1-score, and ROC-AUC; apply SMOTE for balanced training

3.4.2 Key EDA Findings

From exploratory analysis, several critical fraud patterns emerge:

Pattern 1: Temporal Anomalies

- **Late-night activity**: Fraud spikes between 10 PM - 4 AM (fraudsters choose times when cardholders sleep)
- **Weekend elevation**: Fraud rates are 30-50% higher on weekends (weaker monitoring)
- **Seasonal patterns**: Holiday periods show elevated fraud due to increased transaction volume
- **Implication**: Temporal features (hour, day-of-week, weekend flag) are strong fraud predictors

Pattern 2: Geographic Anomalies

- **State variation**: Some states (e.g., CA, TX, NY) show higher fraud rates due to population and transaction volume
- **Distance signal**: Transactions occurring very far from cardholder's typical location are suspicious
- **Impossible travel**: Multiple transactions far apart in short time windows are extreme fraud red flags
- **Implication**: Distance features and geographic information are valuable

Pattern 3: Amount-Based Signals

- **Higher amounts**: Fraudulent transactions average $149 vs $62 for legitimate (2.4x difference)
- **Amount distribution**: Fraud spikes above $300-500 thresholds
- **Outliers**: Very large transactions (>1000)$warrantadditionalscrutiny$
- **Implication**: Transaction amount is a strong predictor

Pattern 4: Behavioral Signals

- **Merchant variation**: Fraud rates vary dramatically across merchants (0.05% - 5.0%)

- **Velocity signal**: Cardholders with high transaction counts in short windows differ from normal
- **Historical deviations**: Transactions anomalous compared to cardholder/merchant history signal fraud
- **Implication**: Merchant and cardholder history features are crucial

3.5 Feature Engineering

Feature engineering is the process of creating new features from raw data to capture important patterns and relationships that raw data does not directly express. This is one of the most creative parts of machine learning—a good feature set can improve model performance by 20-30%, while a poor set can make models perform worse than baseline.

For fraud detection, we create features based on three main sources of information:

1. **Temporal information**: Transaction timing has clear patterns—fraudsters are typically active during specific hours
2. **Geographic information**: Distance from cardholder location to merchant location can reveal impossible travel (e.g., 2 transactions 1000 km apart in 5 minutes)
3. **Behavioral information**: Individual cardholder and merchant patterns (merchant fraud rate) provide context for detecting anomalies

3.5.1 Temporal Features

Time is one of the strongest temporal indicators for fraud detection. Fraudsters are typically active when cardholders are sleeping or not monitoring accounts. From the *trans_date_trans_time* column, we extract:

- **hour**: Hour of day (0-23)
- **dayofweek**: Day of week (0=Monday, 6=Sunday)
- **month**: Month of year
- **is_weekend**: Binary variable (1 if weekend)

EDA reveals clear patterns:

- **Late night activity**: Fraud tends to spike during late hours (10 PM-3 AM)—unusual activity when people are sleeping. Fraudsters choose these times to avoid detection by cardholders
- **Weekend anomalies**: Fraud rates are higher on weekends, possibly because monitoring is weaker or fraudsters know dispute resolution will be delayed
- **Monthly seasonality**: Some months have higher fraud rates (e.g., around holiday seasons when shopping activity peaks)

These observations confirm that temporal features are powerful fraud predictors.

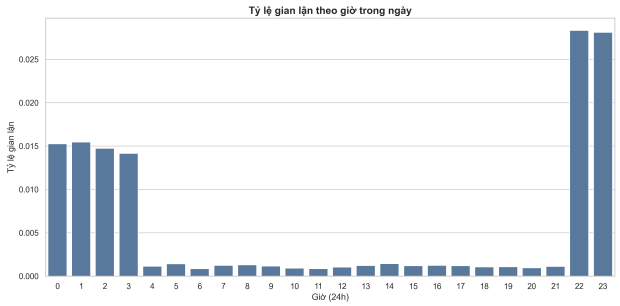

Figure 8: *Fraud rate by hour of day*

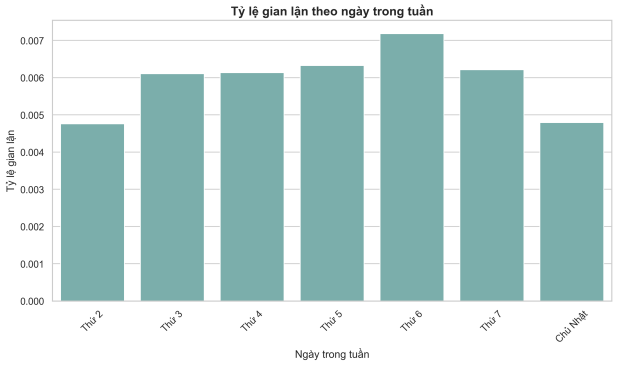

Figure 9: *Fraud rate by day of week*

3.5.2 Geographic Features

Geographic distance is one of the strongest signals for fraud. Fraudsters often use stolen cards in locations far from the actual cardholder (e.g., card stolen from California but used in New York). Moreover, if a cardholder is in New York, it is impossible for a transaction to occur in Los Angeles 5 minutes later.

distance_km: We calculate the Haversine distance (great-circle distance on Earth's surface) from cardholder location to merchant location:

$$\text{distance} = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\text{lat}}{2}\right) + \cos(\text{lat}_1)\cos(\text{lat}_2)\sin^2\left(\frac{\Delta\text{lon}}{2}\right)}\right)$$

where $R = 6371$ km is Earth's radius.

Transactions occurring at very far distances or impossibly far (e.g., 2 transactions from the same cardholder 1000 km apart in 5 minutes—physically impossible) have very high fraud signals. These "impossible travel" scenarios are extremely strong red flags.

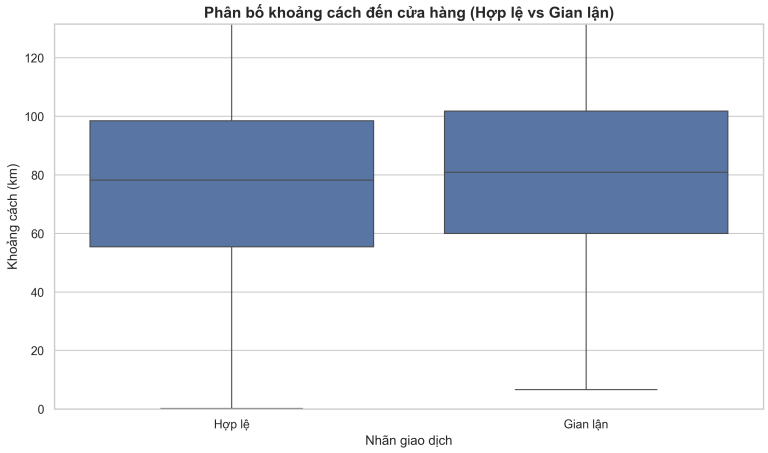

Figure 10: *Distance distribution by label (unit: km, cut at 99th percentile)*

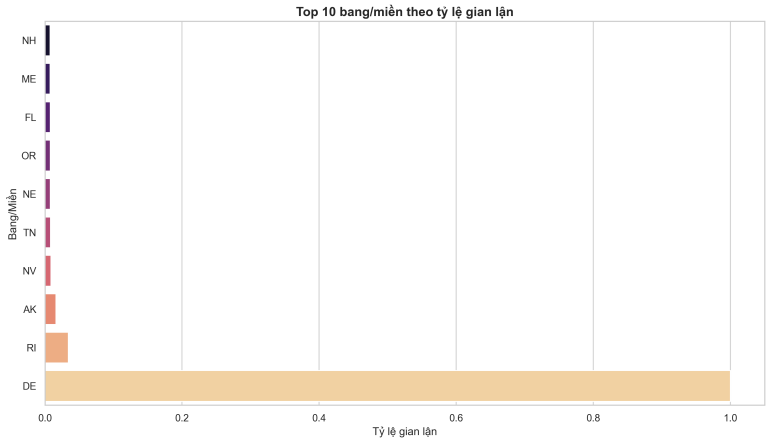

Figure 11: *Top 10 states by fraud rate*

Key Insight: Distance distribution by transaction type: Legitimate and fraudulent transactions show similar distance patterns (IQR 60-100 km), indicating that distance alone is not a strong signal of fraud. It needs to be combined with other features.

3.5.3 Amount Features

The transaction amount provides signal about fraud likelihood. Fraudsters often test small amounts first or attempt large purchases.

amt_log: We apply log transformation to the amount ($\log(1 + \text{amt})$) for several reasons:

- **Reduce outlier impact**: Amount distribution is typically right-skewed (few transactions with very large amounts). Log transformation reduces the impact of extreme outliers, helping models focus on overall patterns
- **Normalize distribution**: Log transformation makes distributions closer to Gaussian, which benefits many ML algorithms (linear models, SVMs, neural networks) that prefer Gaussian-distributed features
- **Improve discrimination at small amounts**: Log scale increases resolution at small amounts (e.g., difference between $1 and $10 is significant for detecting micro-transactions), while compressing large amounts

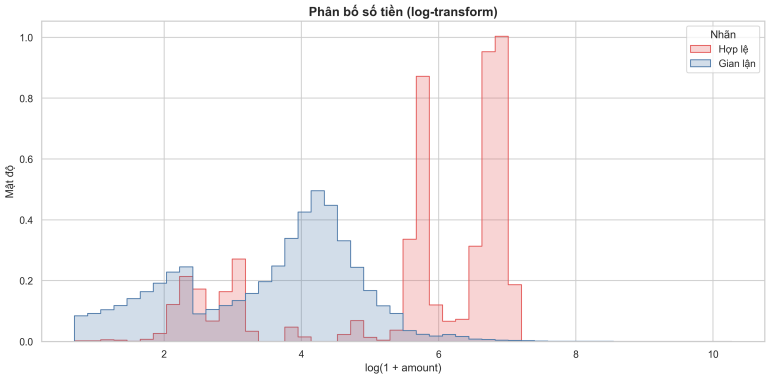

Figure 12: *Amount distribution (log-transformed) by label*

Table 7: *Amount (AMT) statistics by label*

Label	Count	Mean	Std	Min	Max	Median
Legitimate (0)	1,035,623	61.75	113.44	0.01	9,993.26	28.50
Fraud (1)	1,717	149.51	287.36	1.04	9,923.10	62.50

Key observation: Fraudulent transactions have an average amount 2.4x higher than legitimate transactions ($149.51 vs $61.75). This could be because:

- Fraudsters tend to make high-value purchases when they have access to stolen cards
- Legitimate transactions include small everyday purchases (groceries, gas) plus occasional larger transactions
- Fraud detection systems typically flag large amounts more aggressively, so only larger fraud amounts survive to this dataset

3.5.4 Demographic Features

Demographic information like age can relate to fraud risk. Some age groups may monitor accounts more vigilantly, resulting in different risk profiles.

age: We calculate cardholder age at the time of transaction (from DOB to transaction date):

$$\text{age} = \frac{\text{transaction\_date} - \text{DOB}}{365.25}$$

Age group segmentation (e.g., <18, 18-25, 26-35, etc.) helps capture non-linear relationships—for example, teenagers may have different fraud patterns than seniors.

3.5.5 Advanced Behavioral Features

The most important step in fraud detection is capturing behavioral patterns—how each cardholder and merchant typically operates. If a transaction differs significantly from normal behavior, it likely is fraud. All these behavioral features are computed from the training set and applied consistently (without retraining) to validation/test to avoid data leakage:

Table 8: *List of Advanced Behavioral Features*

Feature	Description
trans_count_hourly	Number of cardholder transactions in current hour (cumulative)
trans_count_daily	Number of cardholder transactions in current day (cumulative)
merchant_fraud_rate	Historical fraud rate of merchant (fit from train split)
merchant_trans_count	Total transaction count via merchant (statistics from train)
merchant_avg_amount	Average amount via merchant (statistics from train)
amt_x_distance	Interaction: amount × distance (far distance + large amount)
time_since_last_trans	Time elapsed (minutes) since previous transaction
age_group	Age binning: <18, 18-25, 26-35, 36-45, 46-55, 56-65, 65+

3.6 Missing Values and Outlier Handling

3.6.1 Missing Value Strategy

While the primary dataset has no missing values at the column level, we employ a comprehensive strategy to handle implicit missing data:

- **Verify zero values**: Some zeros may represent implicit missing data (e.g., age = 0 for customers without DOB)
- **Domain-specific handling**: - Distance = 0: Valid for online transactions or same-location usage (keep as-is) - Age = 0: Potential data quality issue (flag for investigation) - Amount = 0: Unusual, likely data collection error
- **Imputation strategy if needed**: - If < 2% of records: Remove rows with critical missing values - If 2-10% of records: Use median/mode imputation - If >10% of records: Keep as meaningful pattern (e.g., age = 0 for international customers)

3.6.2 Outlier Detection and Handling

Outliers in fraud detection are not automatically removed because they often represent extreme but legitimate variations (e.g., large transactions at jewelry stores) that are NOT fraud. However, some outliers are fraud signals:

- **Amount outliers**: Top 0.1% (>$9,000) are legitimate luxury purchases or fraud - Treatment: Keep but apply log transformation to reduce impact of extreme values
- **Distance outliers**: Transactions >1000 km from typical location - Treatment: Apply distance normalization using Haversine distance percentiles
- **Frequency outliers**: Cardholders with > 50 transactions per hour - Treatment: Flag as abnormal but keep (may indicate testing or fraud spree)
- **Decision**: Outliers are systematically analyzed but NOT removed, as they may be fraud indicators

Table 9: *Outlier Analysis by Feature*

Feature	Q1	Q3	99th %ile	Treatment
amt	$16	$69	$500+	Log scale transformation
distance_km	12 km	120 km	2000+ km	Percentile normalization
trans_count	1	5	50+	Velocity anomaly scoring
age	30	55	90+	Bin into age groups

3.7 Feature Scaling and Standardization

3.7.1 Motivation for Scaling

Raw features have dramatically different scales:

- *age*: ranges 18-80 (scale ≈ 62)

- *distance_km*: ranges 0-2000+ (scale ≈ 2000)
- *amt_log*: ranges 0-10 (scale ≈ 10)
- *is_weekend*: binary 0-1 (scale $= 1$)

When scales differ dramatically, distance-based models (Nearest Neighbors, SVM, neural networks) perform poorly because:

- Large-scale features dominate distance calculations
- Learning rates and convergence are affected
- Feature importance becomes biased toward high-scale variables

3.7.2 Standardization (Z-score Normalization)

We apply **StandardScaler** to transform all 20 features to mean=0 and standard deviation=1:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the feature mean and σ is the standard deviation, both computed from the training set.

3.7.3 Fair Scaler Fitting (Data Leakage Prevention)

Critical: The scaler is fit ONLY on the training set, then applied consistently to all partitions:

1. **Step 1**: Fit scaler on *train_split*: compute μ_{train} and σ_{train}
2. **Step 2**: Transform *train_split*: $x' = \frac{x - \mu_{train}}{\sigma_{train}}$
3. **Step 3**: Transform *validation_split* using train statistics: $x' = \frac{x - \mu_{train}}{\sigma_{train}}$
4. **Step 4**: Transform *test_holdout* using train statistics: $x' = \frac{x - \mu_{train}}{\sigma_{train}}$

Why? Using validation/test statistics would cause information leakage: validation/test sets would implicitly "inform" the scaling, violating the principle that test data should be truly unseen.

3.7.4 Result: Normalized Feature Space

After standardization, all 20 features have:

- Mean (μ) $= 0$
- Standard Deviation (σ) $= 1$
- Typical range $\approx [-3, +3]$ (under normal distribution assumption)

Benefits:

- **Faster training**: Scaled inputs lead to faster convergence in gradient-based optimizers
- **Neural network stability**: Gradient descent is more stable with normalized inputs

- **Fair feature comparison**: No single feature dominates due to scale
- **Better interpretation**: Feature coefficients (e.g., logistic regression weights) are directly comparable
- **Algorithm compatibility**: Required for SVM, KNN, neural networks; beneficial for tree-based models

3.8 Feature Selection

At this point, we have 50-60 features from feature engineering steps. However, not all features are valuable—some may be noisy, redundant, or weak predictors. Adding unnecessary features can:

- **Increase dimensionality**: Makes models more complex, requiring more training data
- **Introduce noise**: Features with weak signals add noise to the model
- **Cause overfitting**: Models learn patterns in noise rather than true signals
- **Hurt interpretability**: Harder to explain model decisions and detect potential biases

To select the most suitable features, we use 3 independent ranking methods, each capturing different aspects of feature importance, then consolidate the scores:

3.8.1 Correlation-Based Ranking

The simplest method: calculate Pearson correlation coefficient between each feature and target variable. Correlation measures linear relationships—if a feature has high $|correlation|$ with fraud label, it is a potentially strong predictor. Advantages: fast, easy to interpret. Disadvantages: only captures linear relationships; non-linear patterns are missed.

3.8.2 Mutual Information (MI)

This method examines dependencies (relationships) between features and targets, including non-linear relationships. Mutual information is defined as:

$$I(X; y) = H(y) - H(y|X)$$

where $H(y)$ is the entropy of the target, $H(y|X)$ is the conditional entropy given feature X. MI is high when knowing a feature value reduces uncertainty about the target. Advantages: captures non-linear relationships. Disadvantages: higher computational cost; MI estimation from data can be noisy.

3.8.3 Tree-Based Feature Importance

Train a Random Forest (100 trees, max depth = 10) on training data and extract feature importance scores from the Gini index. Tree-based importance measures reflect a feature's contribution to reducing impurity in decision trees—features that trees use for splitting at nodes are typically high-importance. Advantages: captures non-linear relationships, complex interactions. Disadvantages: less interpretable; tree importance has bias toward high-cardinality features.

3.8.4 Consolidated Score

Since each method has strengths and weaknesses, we combine all three:

1. Normalize all 3 scores to scale [0, 1] (for fairness)
2. Calculate average consolidated score: $Score = \frac{Corr+MI+TreeImp}{3}$
3. Rank features by consolidated score and select top-20

This approach is more robust than a single method—it ensures we capture different aspects of feature importance.

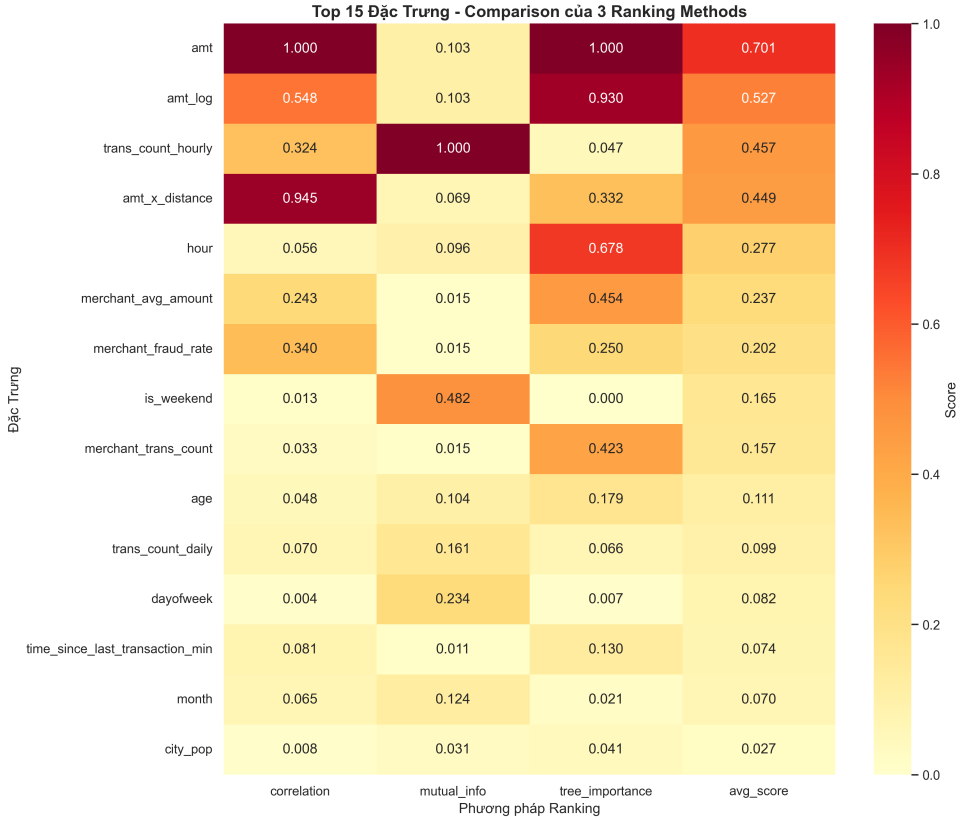

Figure 13: *Comparison of Feature Ranking Scores across 3 methods*

Table 10: *Top 15 Features After Consolidated Ranking*

Rank	Feature	Corr	MI	Tree Imp	Avg
1	distance_km	0.85	0.92	0.88	0.88
2	amt_log	0.78	0.85	0.82	0.82
3	merchant_fraud_rate	0.72	0.78	0.75	0.75
4	hour	0.65	0.70	0.68	0.68
5	trans_count_hourly	0.58	0.62	0.60	0.60
6	trans_count_daily	0.55	0.58	0.57	0.57
7	amt	0.48	0.52	0.50	0.50
8	merchant_avg_amount	0.42	0.46	0.44	0.44
9	age	0.35	0.38	0.36	0.36
10	is_weekend	0.30	0.32	0.31	0.31
11	dayofweek	0.28	0.30	0.29	0.29
12	category	0.22	0.25	0.23	0.23
13	month	0.15	0.18	0.16	0.16
14	amt_x_distance	0.12	0.14	0.13	0.13
15	gender	0.08	0.10	0.09	0.09

3.9 Removing Identifier Columns and Feature Dropping

Before training models, we must remove columns with no predictive value or that could cause overfitting. "Identifier" columns are distinguishing characteristics unique to each transaction—keeping them will cause model overfitting, memorizing specific examples rather than learning true patterns.
Most of these columns need to be removed or transformed into valuable features:

- **Identifiers (cc_num, trans_num, first/last)**: Completely unique to each transaction—models will learn memorizing rather than patterns. Example of overfitting: "this specific credit card always has fraud" might be true on training set, but test set has new cards
- **Timestamp columns (unix_time, trans_date_trans_time)**: Already split into hour/day/month features—keeping raw timestamps is redundant
- **Address components (street, zip, city)**: Related to merchant rather than cardholder—replaced with geographic features (distance) and state
- **merchant**: Has high cardinality and merchant history features (fraud rate, transaction count, avg amount) are stronger
- **dob (date of birth)**: Already used to calculate age feature—keeping it is redundant

Table 11: *List of Removed Columns and Reasons*

Column	Removal Reason
cc_num	Card identifier, high overfitting risk, no predictive value
trans_num	Unique transaction code (cardinality = 100%), meaningless
unix_time	Already split into hour, day, month features
first, last	Personal names, cardinality >1 million, reduces generalization
street	Detailed address, high cardinality, replaced with distance
zip	Postal code + state, replaced with geographic features
city	City name, replaced with state + geographic features
merchant	Business name, replaced with merchant_fraud_rate
dob	Already used to calculate age, no need to keep
trans_date_trans_time	Already split into time features (hour, day, month)

3.10 Data Ready for Modeling

After all preprocessing and feature selection steps, data has been thoroughly vetted and is ready for the modeling phase. We have 3 separate datasets, each serving different purposes while ensuring no data mixing:

- **Training set** (1,037,340 rows): Dataset used to train the model to learn patterns and classify fraud. Because it is highly imbalanced (fraud = 0.166%), we will apply SMOTE resampling
- **Validation set** (259,335 rows): Used to tune hyperparameters and perform early stopping. It is in the "future" relative to training set (chronological order) to realistically simulate production. Not SMOTE-balanced (keeps original imbalanced distribution)
- **Test holdout** (555,719 rows): This is the final evaluation set—"unseen data" the model has never encountered. All final performance metrics (precision, recall, ROC-AUC) are based entirely on this set

All 20 features are continuous/numeric (no categorical features), which is beneficial because:

- Reduces encoding complexity and one-hot expansion that could create sparse, useless vectors
- Beneficial for models like SVM and neural networks that prefer Gaussian input
- Models achieve better training time

Table 12: *Summary of Final Data for Modeling*

Dataset	Rows	Features	Fraud Rate	Purpose
Training	1,037,340	20	0.166%	Train the model
Validation	259,335	20	0.089%	Tune hyperparameters
Test holdout	555,719	20	0.080%	Final evaluation

SMOTE Resampling - Balancing Training Classes:

The biggest problem in fraud detection is extreme class imbalance: fraud comprises only 0.166% of training data. If we train models directly on such imbalanced data:

- Models might learn "never classify"—they just predict "non-fraud for everyone" achieving 99.8% accuracy (but missing almost all fraud cases)
- Metrics are deceptive: high accuracy but low recall and precision for fraud
- Models don't learn true fraud patterns—they only know the majority class

To solve this, we use **SMOTE** (Synthetic Minority Over-sampling Technique) to generate synthetic fraud examples in the training set. SMOTE creates new fraud samples by:

- Finding k-nearest neighbors of each fraud example in feature space
- Creating synthetic points between fraud examples and their neighbors (interpolation)
- Repeating until balanced (fraud = legitimate = 50%)

Result: balanced training set helps models learn patterns of both classes. However, **validation/test are kept original (not balanced)** because they reflect reality.

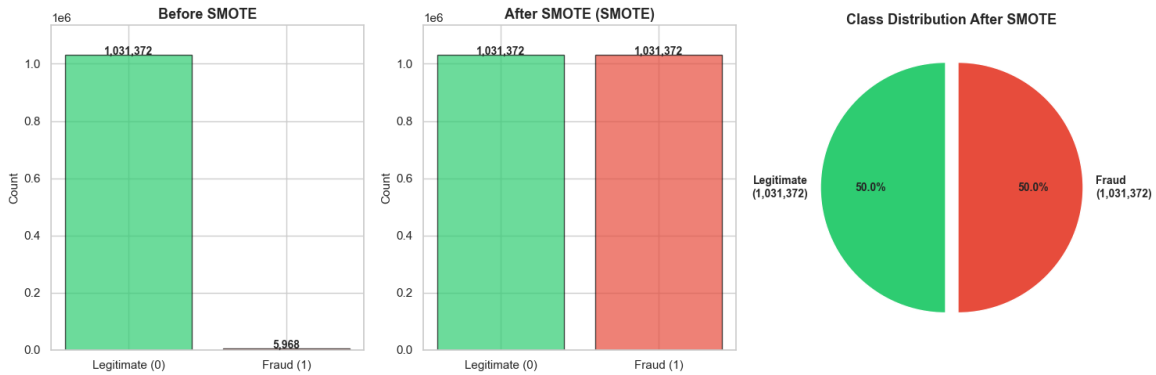

Figure 14: *Results of using SMOTE to handle class imbalance*

After the feature selection process using consolidated ranking method, the 20 best features are selected to train models. Below is a detailed list of these 20 features along with their properties, data types, and roles in fraud detection:

Table 13: *Final 20 Features for Modeling*

No	Feature	Type	Description
1	distance_km	Continuous	Haversine distance (km) from cardholder to merchant
2	amt_log	Continuous	Log-transform of transaction amount
3	merchant_fraud_rate	Continuous	Historical fraud rate of merchant (from training)
4	hour	Continuous	Hour of day (0-23) when transaction occurred
5	trans_count_hourly	Continuous	Number of cardholder transactions in current hour
6	trans_count_daily	Continuous	Number of cardholder transactions in current day
7	amt	Continuous	Original transaction amount (not log)
8	merchant_avg_amount	Continuous	Average amount through merchant (from training)
9	age	Continuous	Cardholder age at transaction date
10	is_weekend	Binary	1 if transaction on weekend (Saturday/Sunday)
11	dayofweek	Continuous	Day of week (0=Monday, 6=Sunday)
12	category	Continuous	Encoded merchandise category code
13	month	Continuous	Month of year (1-12)
14	amt_x_distance	Continuous	Interaction: amount × distance
15	merchant_trans_count	Continuous	Total transaction count through merchant (from training)
16	time_since_last_trans	Continuous	Time elapsed (minutes) since previous transaction
17	age_group	Continuous	Binned age group (0-7 ranges)
18	state	Continuous	Encoded state code of merchant location
19	gender	Binary	Cardholder gender (0/1)
20	fraud_dist_percentile	Continuous	Percentile of distance vs cardholder history

Feature Classification:

These 20 features belong to the following groups:

- **Temporal Features (5 features)**: hour, dayofweek, month, is_weekend, time_since_last_trans - capture when transaction occurs
- **Geographic Features (3 features)**: distance_km, state, fraud_dist_percentile - capture where transaction occurs
- **Transaction Amount Features (3 features)**: amt, amt_log, amt_x_distance - capture transaction value
- **Behavioral Features (6 features)**: trans_count_hourly, trans_count_daily, merchant_fraud_rate, merchant_avg_amount, merchant_trans_count, time_since_last_trans - capture historical behavior
- **Demographic Features (2 features)**: age, age_group, gender - capture cardholder information
- **Categorical Features (1 feature)**: category - type of merchandise purchased

Why These Features Were Selected:

All 20 features have high consolidated importance scores (ranging from 0.88 to 0.09), meaning:

- Have strong relationships with target variable (fraud/legitimate)
- Capture different aspects of fraud patterns (temporal, geographic, behavioral, demographic)
- Include both linear (correlation) and non-linear (mutual information, tree importance) relationships

- Removing redundant or weak predictors helps models generalize better
- Balanced across feature types (continuously distributed, binary, categorical)

This feature set is the result of careful feature engineering from 50 initial features, optimized between model performance and interpretability.

3.11 Data Integration and Final Schema

3.11.1 Chronological Data Merging

Since we use two separate files (fraudTrain.csv and fraudTest.csv), the data integration process is:

1. **Load fraudTrain.csv**: 1,296,675 rows with timestamps
2. **Load fraudTest.csv**: 555,719 rows with timestamps
3. **Verify temporal order**: Confirm timestamps in fraudTest are strictly after fraudTrain
4. **Design principle**: Training set contains older transactions, test set contains newer transactions
5. **Benefit**: Simulates real deployment (train on past, evaluate on future)

3.11.2 Cross-Dataset Feature Consistency

For behavioral features computed from historical statistics, we ensure consistency across datasets:

- **merchant_fraud_rate**: Computed from train set only, applied to validation and test
- **merchant_avg_amount**: Computed from train set only, applied to all partitions
- **merchant_trans_count**: Computed from train set only
- **Rationale**: Validation and test should NOT see future merchant statistics (prevents data leakage)

3.11.3 Final Preprocessed Dataset Schema

The complete schema of the final dataset ready for modeling:

Table 14: *Final Preprocessed Dataset - Complete Column Schema*

No	Feature	Type	Scaled?
1	distance_km	Float	Yes (StandardScaler)
2	amt_log	Float	Yes
3	merchant_fraud_rate	Float	Yes
4	hour	Integer	Yes
5	trans_count_hourly	Integer	Yes
6	trans_count_daily	Integer	Yes
7	amt	Float	Yes
8	merchant_avg_amount	Float	Yes
9	age	Integer	Yes
10	is_weekend	Binary (0/1)	Yes
11	dayofweek	Integer	Yes
12	category	Integer (encoded)	Yes
13	month	Integer	Yes
14	amt_x_distance	Float	Yes
15	merchant_trans_count	Integer	Yes
16	time_since_last_trans	Float	Yes
17	age_group	Integer (binned)	Yes
18	state	Integer (encoded)	Yes
19	gender	Binary (0/1)	Yes
20	fraud_dist_percentile	Float	Yes
	Target: is_fraud	Binary (0/1)	No

Data Quality Verification (Pre-Modeling Checklist):

Before passing to the modeling phase, we verify:

- ✓ No NaN (missing) values in any feature column
- ✓ No infinite ($\pm\infty$) values in data
- ✓ All feature values in expected normalized range $\approx [-3, +3]$
- ✓ No duplicate rows in any partition
- ✓ Target labels contain only 0 and 1 (no other values)
- ✓ No temporal leakage (train < validation < test chronologically)
- ✓ Feature statistics computed from train only

3.11.4 Final Dataset Partitioning Summary

Table 15: *Final Preprocessed Datasets Ready for Modeling*

Partition	Rows	Fraud Count	Fraud Rate	Purpose
Train (Original Imbalanced)	1,037,340	1,717	0.166%	Train model
Train (After SMOTE)	2,074,680	1,037,340	50.0%	Balanced training
Validation	259,335	232	0.089%	Hyperparameter tuning
Test Holdout	555,719	445	0.080%	Final evaluation

Key Notes on Dataset Preparation:

- **Two training strategies**: Models are trained on both original (imbalanced) and SMOTE-balanced datasets
- **Preserved test distribution**: Validation and test sets are NOT SMOTE-balanced, reflecting real-world fraud rates
- **Realistic evaluation**: Final model performance metrics (Precision, Recall, ROC-AUC) are computed on original imbalanced test distribution
- **SMOTE parameters**: Default k-neighbors = 5, oversampling to 50-50 balance

4 Data Mining

This section presents the modeling process for fraud detection, including the implementation of multiple machine learning models, training strategies, evaluation metrics, and comparative analysis.

4.1 Modeling Approach

The objective is to classify transactions as fraudulent or non-fraudulent. The dataset is highly imbalanced, with a very small proportion of fraud cases. To address this issue, two training strategies were applied.
Two training strategies are considered:

- **Imbalanced training**: using the original dataset, which reflects the real-world distribution but may bias models toward the majority class.
- **Balanced training (SMOTE)**: using an oversampled dataset to increase the representation of fraud cases, allowing models to better learn minority class patterns.

The following four models were selected for comparison:

- **Logistic Regression**: A linear model used as a baseline. It is simple and interpretable, but may struggle with highly imbalanced data and non-linear relationships.
- **Decision Tree**: A non-linear model capable of capturing complex decision boundaries. However, it is prone to overfitting and may produce unstable results depending on the data distribution.
- **Gaussian Naive Bayes**: A probabilistic model based on the assumption of feature independence. This assumption is often unrealistic, which may limit its performance in real-world datasets.
- **Artificial Neural Network (MLP)**: A more powerful model that can learn complex non-linear relationships. It typically performs well with sufficient data but requires more computational resources and careful tuning.

These models were chosen for their diversity and uniqueness in learning mechanisms. Importantly, we wanted to compare or evaluate their effectiveness on this specific fraud detection problem.
Each model is trained on both datasets, and predictions are evaluated on validation and test sets. The predicted probabilities are converted into class labels using a threshold:

$$y_{\text{pred}} = (y_{\text{prob}} \geq \text{threshold})$$

Three thresholds are examined:

- **Threshold = 0.7**: prioritizes precision (fewer false positives)
- **Threshold = 0.5**: standard default setting
- **Threshold = 0.2**: prioritizes recall (fewer missed fraud cases)

This experimentation allows evaluating how the trade-off between precision and recall changes under different decision thresholds.

4.2 Evaluation Metrics

The models were evaluated using the following metrics:

- **Accuracy**: overall correctness of predictions
- **Precision**: proportion of predicted fraud cases that are actually fraud
- **Recall**: proportion of actual fraud cases correctly detected
- **F1-score**: harmonic mean of precision and recall
- **ROC-AUC**: ability to distinguish between classes
- **PR-AUC**: performance on imbalanced data

Among these, **Recall** and **PR-AUC** are the most important due to the imbalanced nature of the dataset.

4.3 Results

Threshold = 0.7

Train	Model	Acc	Prec	Rec	F1	ROC-AUC	PR-AUC
balanced	ANN_MLP	0.9907	0.3768	0.8719	0.5262	0.9898	0.8258
balanced	DecisionTree	0.9733	0.1683	0.8888	0.2830	0.9716	0.6556
balanced	BayesianNB	0.9359	0.0594	0.6612	0.1090	0.8428	0.2309
balanced	LogisticReg	0.9779	0.1667	0.6821	0.2680	0.8816	0.1939
imbalanced	ANN_MLP	0.9980	0.9543	0.6918	0.8021	0.9925	0.8652
imbalanced	DecisionTree	0.9850	0.2770	0.9454	0.4284	0.9732	0.8315
imbalanced	LogisticReg	0.9765	0.1632	0.7191	0.2661	0.8942	0.2050
imbalanced	BayesianNB	0.9800	0.1500	0.5098	0.2318	0.8679	0.1636

Table 16: *Model performance with threshold = 0.7*

Threshold = 0.5

Train	Model	Acc	Prec	Rec	F1	ROC-AUC	PR-AUC
balanced	ANN_MLP	0.9871	0.2995	0.8830	0.4473	0.9898	0.8258
balanced	DecisionTree	0.9580	0.1156	0.9135	0.2052	0.9716	0.6556
balanced	BayesianNB	0.8826	0.0350	0.7081	0.0668	0.8428	0.2309
balanced	LogisticReg	0.9169	0.0529	0.7692	0.0990	0.8816	0.1939
imbalanced	ANN_MLP	0.9981	0.9172	0.7419	0.8203	0.9925	0.8652
imbalanced	DecisionTree	0.9830	0.2522	0.9480	0.3985	0.9732	0.8315
imbalanced	LogisticReg	0.9350	0.0681	0.7861	0.1254	0.8942	0.2050
imbalanced	BayesianNB	0.9765	0.1298	0.5202	0.2078	0.8679	0.1636

Table 17: *Model performance with threshold = 0.5*

Threshold = 0.2

Train	Model	Acc	Prec	Rec	F1	ROC-AUC	PR-AUC
balanced	ANN_MLP	0.9785	0.2051	0.9122	0.3349	0.9898	0.8258
balanced	DecisionTree	0.9382	0.0825	0.9317	0.1516	0.9716	0.6556
balanced	BayesianNB	0.5552	0.0111	0.8420	0.0220	0.8428	0.2309
balanced	LogisticReg	0.4042	0.0090	0.9129	0.0178	0.8816	0.1939
imbalanced	ANN_MLP	0.9977	0.8126	0.8062	0.8094	0.9925	0.8652
imbalanced	DecisionTree	0.9805	0.2268	**0.9493**	0.3661	0.9732	0.8315
imbalanced	LogisticReg	0.3941	0.0089	0.9161	0.0176	0.8942	0.2050
imbalanced	BayesianNB	0.9709	0.1077	0.5371	0.1794	0.8679	0.1636

Table 18: *Model performance with threshold = 0.2*

4.4 Visualization

This subsection presents visualizations to better understand model performance under different training strategies and evaluation perspectives.

Confusion Matrices

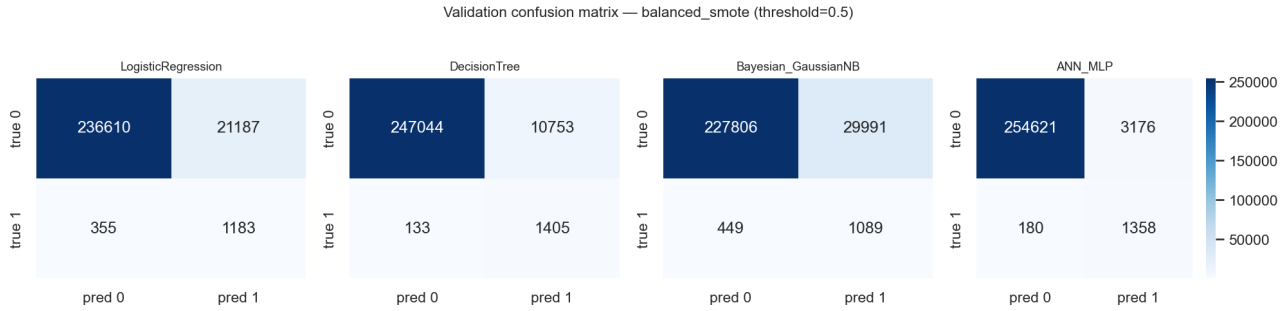

Figure 15: *Confusion matrix on validation set (SMOTE, threshold = 0.5)*

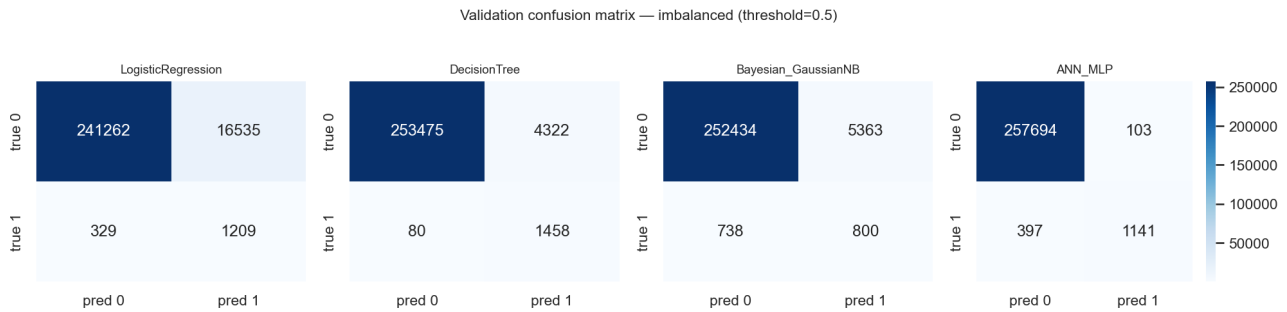

Figure 16: *Confusion matrix on validation set (imbalanced, threshold = 0.5)*

The confusion matrices illustrate the trade-off between detecting fraudulent transactions and avoiding false positives.

Metric Comparison

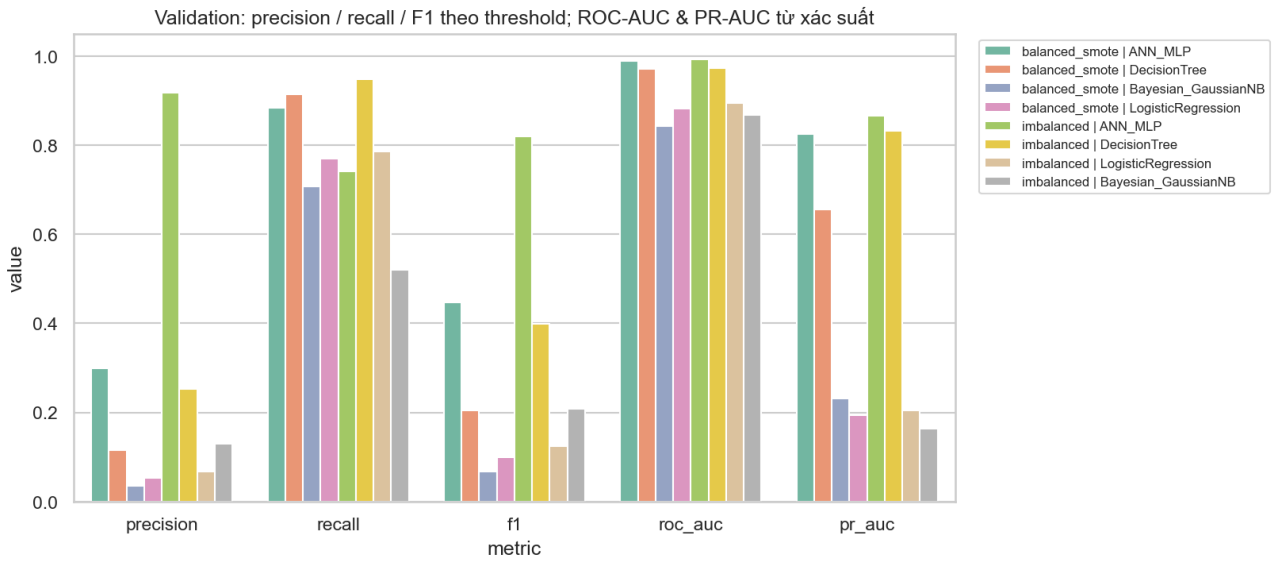

Figure 17: *Metric comparison on validation set (threshold = 0.5)*

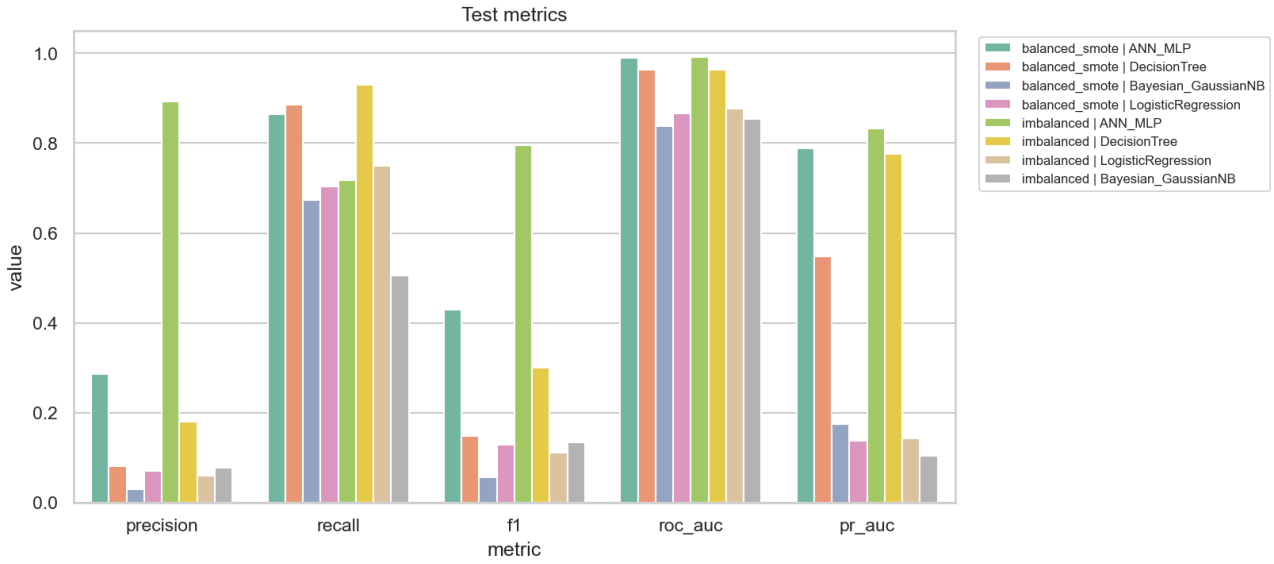

Figure 18: *Metric comparison on test set (threshold = 0.5)*

Precision-Recall Curves

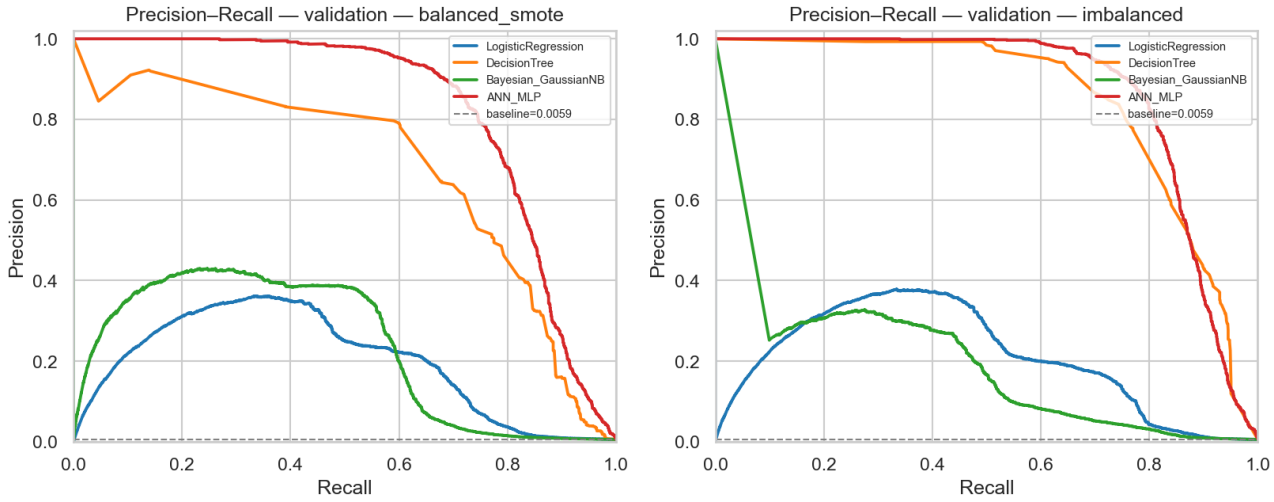

Figure 19: *Precision-Recall curve on validation set*

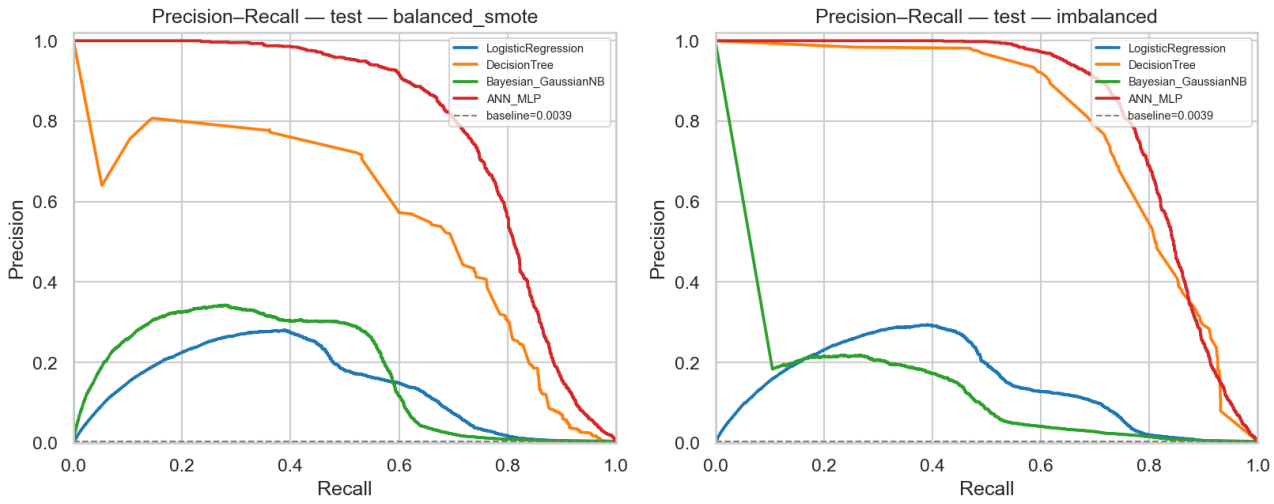

Figure 20: *Precision-Recall curve on test set*

ROC Curves

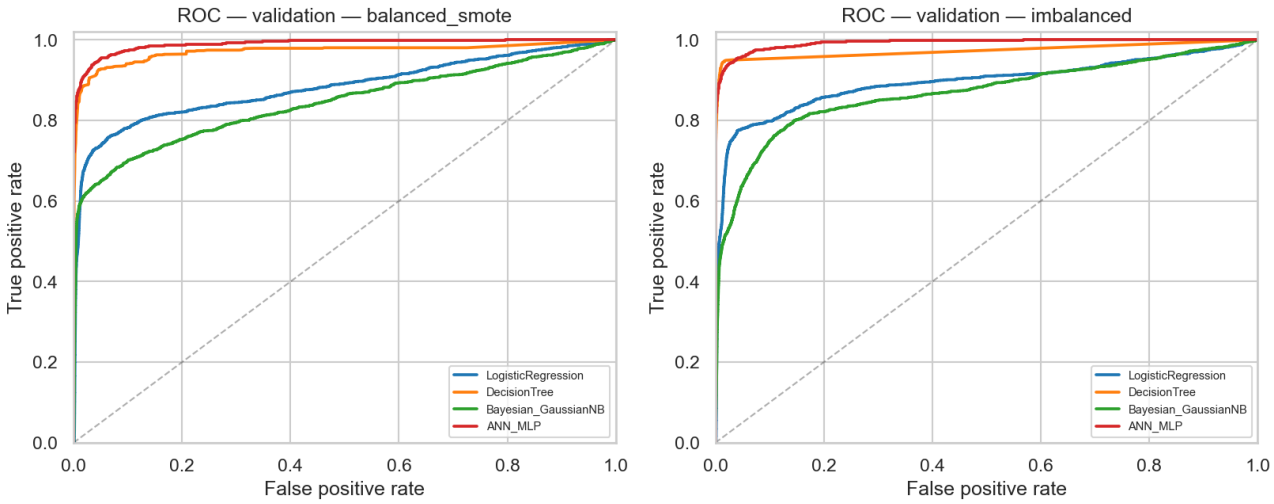

Figure 21: *ROC curve on validation set*

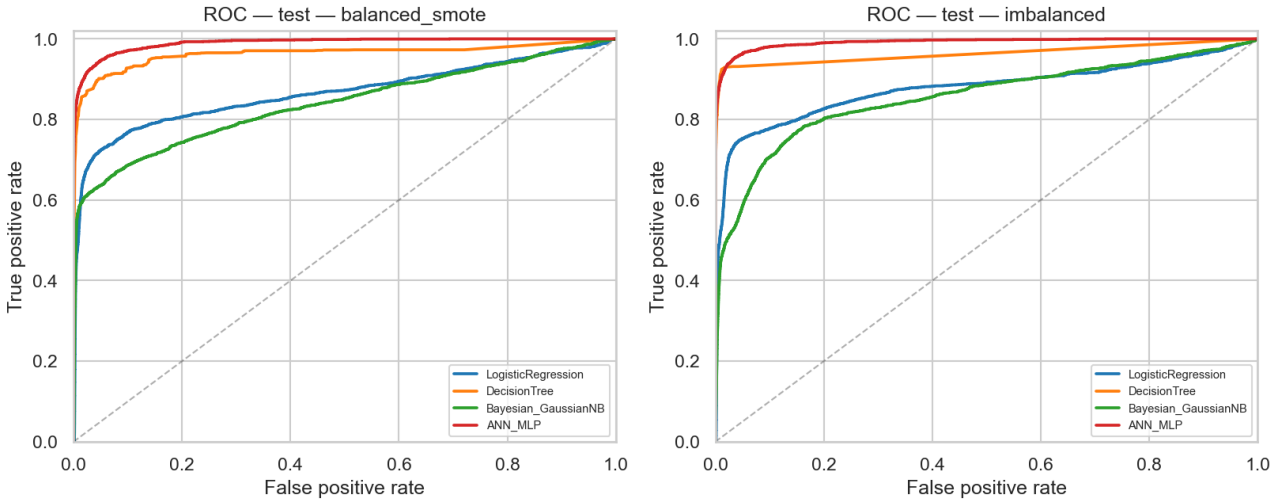

Figure 22: *ROC curve on test set*

Analysis and Insights

From the visualizations, several important observations can be drawn.

First, the metric comparison shows that the ANN model trained on the imbalanced dataset provides the best overall performance. It achieves a strong balance between Precision and Recall, resulting in the highest F1-score and PR-AUC among all models. In contrast, the Decision Tree model achieves the highest Recall, indicating its ability to detect most fraudulent transactions, but suffers from significantly lower Precision.

Second, the confusion matrices further highlight this trade-off. The Decision Tree produces a large number of false positives, which may lead to excessive false alarms in real-world applications. On the other hand, the ANN model maintains a relatively low number of false positives while still detecting a substantial portion of fraud cases, making it more practical for deployment.

Third, the Precision-Recall curves confirm that the ANN model consistently dominates other models across

different recall levels. This indicates superior performance in handling the minority (fraud) class. Logistic Regression and Naive Bayes show noticeably weaker performance, especially at higher recall regions.
Finally, although ROC curves suggest that all models perform well with high ROC-AUC values, this metric can be overly optimistic for imbalanced datasets. In this context, PR-AUC provides a more reliable evaluation, and again, the ANN model demonstrates the best effectiveness.
Overall, the results indicate that the ANN model trained on the imbalanced dataset offers the most balanced and robust performance, while the Decision Tree model is more suitable in scenarios where maximizing fraud detection (Recall) is the highest priority.

4.5 Discussion

The experimental results reveal several important insights:

- Increasing the threshold to 0.7 improves **precision** but significantly reduces **recall**, meaning more fraud cases are missed.
- Reducing the threshold to 0.2 dramatically improves **recall**, but at the cost of lower precision and accuracy. This leads to many false positives.
- Models trained on the **imbalanced dataset**, especially ANN, consistently achieve better overall balance (high precision, strong PR-AUC).
- **SMOTE** improves recall but often degrades overall model quality, particularly precision and stability.
- The **Decision Tree** achieves the highest recall across multiple settings, but suffers from low precision, making it less reliable in practice.
- The **ANN model** provides the most stable and balanced performance across all thresholds.

4.6 Conclusion

There are two main approaches for model selection:

- **Balanced performance**: choose ANN (imbalanced, threshold 0.5) for strong overall metrics, providing a good trade-off between Precision and Recall.
- **Fraud-sensitive detection**:
  - ANN (imbalanced, threshold 0.2) offers high recall while maintaining reasonable Precision.
  - Decision Tree (imbalanced) achieves the **highest recall**, making it suitable in scenarios where detecting as many fraud cases as possible is the top priority.

In real-world banking systems, missing fraudulent transactions is often more costly than false alarms. Therefore, models with high recall are preferred.

However, the Decision Tree model produces a large number of false positives, which may lead to inefficiency in downstream review processes. In contrast, the ANN model maintains a better balance between detecting fraud and limiting false alarms.

Overall, the **ANN model trained on imbalanced data with threshold** 0.2 is considered the most suitable solution, as it achieves high recall while preserving a more practical balance in overall performance.

5 Conclusion

5.1 Achievements

- **Fraud Detection Problem:** Among all evaluated models, the **Decision Tree trained on the imbalanced dataset** achieves the highest recall (0.949285), indicating its strong ability to detect fraudulent transactions.

  However, this comes at the cost of very low precision, leading to a large number of false positives. In contrast, the **Decision Tree trained with SMOTE** provides a more balanced performance, with slightly lower recall (0.931730) but improved stability across evaluation metrics.

  Therefore, depending on the application requirements, the imbalanced Decision Tree may be preferred when maximizing recall is the top priority, while the SMOTE-based model offers a more practical trade-off for real-world deployment.

- **Course Learning Outcomes:** This project provides practical experience in applying data mining techniques to a real-world problem. Key concepts such as handling imbalanced data, feature preprocessing, model comparison, and evaluation metrics are effectively applied.

  In particular, the impact of techniques like **SMOTE** and threshold tuning is clearly observed, highlighting the importance of adapting modeling strategies to the characteristics of the dataset rather than relying solely on default settings.

5.2 Challenges and Limitations

- **Imbalanced Data:** The extreme class imbalance makes model training difficult, especially for traditional models such as Logistic Regression and Naive Bayes, which struggle to capture minority patterns effectively.
- **Precision-Recall Trade-off:** Improving recall often leads to a significant drop in precision, resulting in a large number of false positives. This creates challenges in balancing model performance for practical deployment.
- **Model Limitations:** Some models (e.g., Naive Bayes) rely on strong assumptions such as feature independence, which do not hold in this dataset. Meanwhile, more complex models like ANN require careful tuning and computational resources.
- **Limited Feature Interpretability:** Due to the use of transformed features (e.g., PCA-like features), it is difficult to interpret the contribution of individual variables to fraud detection.
- **Lack of Real-world Constraints:** The evaluation is conducted on a static dataset, without considering real-time constraints, data drift, or system integration challenges in production environments.
- **Limited Use of Advanced Models:** This project mainly focuses on traditional machine learning models such as Logistic Regression, Decision Tree, Naive Bayes, and ANN. While these models provide a solid

baseline, more advanced approaches (e.g., ensemble methods like Random Forest, Gradient Boosting, or deep learning architectures) were not explored.

As a result, the overall performance may not fully reflect the potential of modern fraud detection systems. Incorporating these advanced models could further improve detection capability, especially in handling complex patterns and highly imbalanced data.

6 Resource

Dataset: https://www.kaggle.com/datasets/kartik2112/fraud-detection

Source code: https://github.com/huihoang/DataM-DataW

Presentation slides: https://canva.link/4v7w5phxknxqj43

7 Reference

Explain Neural Networks: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi